

JavaScript: Zero to Hero (Novel to Ninja) Complete Documentation

સરળ ગુજરાતી સમજૂતી અને ઉદાહરણો સાથે (W3Schools, TutorialsPoint & JavaScript.info ના આધારે)

ભાગ ૧: બિગિનર લેવલ કોન્સેપ્ટ્સ (Beginner Level Concepts)

૧. JavaScript નો પરિચય (Introduction to JavaScript)

- વ્યાખ્યા:** JavaScript (JS) એ એક લાઇવેઇટ, ઇન્ટરપ્રેટેડ, અને જસ્ટ-ઇન-ટાઇમ (JIT) કમ્પાઇલ થતી પ્રોગ્રામિંગ લેંગ્વેજ છે. તે મુખ્યત્વે વેબ પેજોને ડાયનેમિક અને ઇન્ટરેક્ટિવ બનાવવા માટે વપરાય છે.
- HTML/CSS સાથે સંબંધ:** HTML વેબ પેજનું માળખું (Structure) બનાવે છે, CSS તેને સુંદરતા (Design) આપે છે, અને JavaScript તેમાં જીવ (Behavior) પૂરે છે.
- નિયમ (Syntax):** JS ના સ્ટેટમેન્ટના અંતે સેમીકોલોન (;) મૂકવો એ સારી આદત છે. JS એ **Case-Sensitive** લેંગ્વેજ છે (એટલે કે myVar અને myvar બંને અલગ ગણાય).

૨. વેરિએબલ્સ (Variables)

વેરિએબલ એટલે ડેટા સ્ટોર કરવા માટેનું એક કન્ટેનર (પાત્ર). JS માં વેરિએબલ ડિક્લેર કરવા માટે ૩ કીવર્ડ્સ છે:

- var:** (જૂની પદ્ધતિ) આ ફંક્શન-સ્કોપ છે અને તેને ફરીથી ડિક્લેર કરી શકાય છે.
- let:** (આધુનિક) આ બ્લોક-સ્કોપ ({ }) છે. તેની વેલ્યુ બદલી શકાય છે.
- const:** (આધુનિક) આ પણ બ્લોક-સ્કોપ છે. એકવાર વેલ્યુ આપ્યા પછી તેને બદલી શકાતી નથી (Constant).

```
// ઉદાહરણ
var name = "રાહુલ"; // જૂની રીત
let age = 25;        // બદલી શકાય તેવો વેરિએબલ
const pi = 3.14;     // અચલ વેરિએબલ
```

૩. ડેટા ટાઇપ્સ (Data Types)

JS માં મુખ્યત્વે બે પ્રકારના ડેટા ટાઇપ્સ હોય છે: **Primitive** અને **Non-Primitive (Reference)**.

Primitive Data Types:

- String:** ટેક્સ્ટ ડેટા, જેને સિંગલ કે ડબલ કોટ્સમાં લખાય. જેમ કે "નમસ્તે".
- Number:** પૂર્ણાંક કે અપૂર્ણાંક સંખ્યાઓ. જેમ કે 45, 99.99
- BigInt:** ખૂબ મોટી સંખ્યાઓ માટે.
- Boolean:** માત્ર બે જ વેલ્યુ: true (સાચું) અથવા false (ખોટું).
- Undefined:** વેરિએબલ ડિક્લેર કર્યો હોય પણ વેલ્યુ ન આપી હોય.

- **Null:** જાણી જોઈને ખાલી (Empty) રાખેલાં વેલ્યુ.
- **Symbol:** યુનિક આઈડેન્ટિફાયર માટે.

Non-Primitive Data Types:

- **Object:** કી-વેલ્યુ પેર (Key-Value pairs) નો સંગ્રહ.

૪. ઓપરેટર્સ (Operators)

- **Arithmetic Operators:** + (સરવાળો), - (બાદબાકી), * (ગુણાકાર), / (ભાગાકાર), % (શેષ), ** (ઘાત).
- **Assignment Operators:** =, +=, -=, *=, /=
- **Comparison Operators:**
 - ==: માત્ર વેલ્યુ ચેક કરે છે (Equal to).
 - ===: વેલ્યુ અને ડેટા ટાઇપ બંને ચેક કરે છે (Strict equal to).
 - !=, !==, >, <, >=, <=
- **Logical Operators:** && (AND - બધા સાચા હોવા જોઈએ), || (OR - ગમે તે એક સાચું હોય તો ચાલે), ! (NOT - ઊલટું કરવું).

૫. કન્ડિશનલ સ્ટેટમેન્ટ્સ (Conditional Statements)

જ્યારે પ્રોગ્રામમાં કોઈ નિર્ણય (Decision) લેવાનો હોય ત્યારે આ વપરાય છે.

- **if સ્ટેટમેન્ટ:** જો કન્ડિશન સાચી હોય તો જ અંદરનો કોડ રન થાય.
- **if...else સ્ટેટમેન્ટ:** જો કન્ડિશન સાચી હોય તો if વાળો ભાગ, નહીંતર else વાળો ભાગ રન થાય.
- **if...else if...else સ્ટેટમેન્ટ:** એક કરતાં વધુ કન્ડિશન ચેક કરવા માટે.
- **switch સ્ટેટમેન્ટ:** એક જ વેરિએબલની ઘણી બધી અલગ-અલગ વેલ્યુ ચેક કરવા માટે (મેનુ જેવું).

```
let marks = 75;
if (marks >= 80) {
  console.log("ગ્રેડ: A");
} else if (marks >= 60) {
  console.log("ગ્રેડ: B"); // આ પ્રિન્ટ થશે
} else {
  console.log("ગ્રેડ: C");
}
```

૬. લૂપ્સ (Loops)

એક જ કોડને વારંવાર રન (Repeat) કરવા માટે લૂપ્સ વપરાય છે.

- **for loop:** જ્યારે આપણને ખબર હોય કે કોડ કેટલી વાર ફેરવવાનો છે.
- **while loop:** જ્યાં સુધી કન્ડિશન સાચી રહે ત્યાં સુધી કોડ ચાલ્યા કરે.
- **do...while loop:** કોડ ઓછામાં ઓછો એકવાર તો રન થશે જ, પછી કન્ડિશન ચેક થશે.

```
// ૧ થી ૩ પ્રિન્ટ કરવા માટે For Loop
for (let i = 1; i <= 3; i++) {
  console.log("નંબર: " + i);
}
```

૭. ફંક્શન્સ - બેઝિક્સ (Functions Basics)

ફંક્શન એટલે કોઈ એક ચોક્કસ કામ કરવા માટે બનાવેલો કોડનો બ્લોક, જેને તમે ગમે ત્યારે ફરી વાપરી (Reuse) શકો છો.

```
// ફંક્શન બનાવવું (Definition)
function sayHello(name) {
  return "નમસ્તે, " + name + "!";
}

// ફંક્શનનો ઉપયોગ કરવો (Calling)
let message = sayHello("અમિત");
console.log(message); // આઉટપુટ: નમસ્તે, અમિત!
```

૮. એરે - બેઝિક્સ (Arrays Basics)

એક જ વેરિએબલમાં એક કરતાં વધુ વેલ્યુની યાદી (List) સ્ટોર કરવા માટે એરે વપરાય છે. એરે ઇન્ડેક્સ ૦ થી શરૂ થાય છે.

```
let fruits = ["સફરજન", "કેળું", "કેરી"];
console.log(fruits[0]); // આઉટપુટ: સફરજન
console.log(fruits.length); // એરેની લંબાઈ આપશે (3)
```

ભાગ ૨: ઇન્ટરમીડિયેટ લેવલ કોન્સેપ્ટ્સ (Intermediate Level Concepts)

૧. ઓબ્જેક્ટ્સ વિગતવાર (Objects in Detail)

ઓબ્જેક્ટ એ પ્રોપર્ટીઝ અને મેથડ્સનું કલેક્શન છે.

```
let car = {
  brand: "Tata",
  model: "Nexon",
  start: function() {
    return this.brand + " કાર ચાલુ થઈ ગઈ.";
  }
};
console.log(car.brand); // પ્રોપર્ટી એક્સેસ (Tata)
console.log(car.start()); // મેથડ કોલ
```

૨. એરે મેથડ્સ (Advanced Array Methods)

- push() / pop():** એરેના અંતમાં આઇટમ ઉમેરવા / છેલ્લી આઇટમ કાઢવા માટે.
- shift() / unshift():** એરેની શરૂઆતમાંથી આઇટમ કાઢવા / શરૂઆતમાં ઉમેરવા માટે.
- map():** એરેના દરેક એલિમેન્ટ પર પ્રોસેસ કરીને નવો એરે બનાવે છે.
- filter():** કન્ડિશન મેચ થતી હોય તેવા એલિમેન્ટ્સ ગાળીને નવો એરે આપે છે.
- reduce():** એરેના બધા એલિમેન્ટ્સ ભેગા કરીને સિંગલ વેલ્યુ (જેમ કે સરવાળો) આપે છે.

```
let numbers = [1, 2, 3, 4];
let doubles = numbers.map(x => x * 2); // [2, 4, 6, 8]
let evens = numbers.filter(x => x % 2 === 0); // [2, 4]
```

૩. સ્કોપ અને હોઇસ્ટિંગ (Scope & Hoisting)

- Scope:** વેરિએબલ કોડમાં ક્યાં એક્સેસ થઈ શકે તે નક્કી કરે છે (Global, Function, અને Block Scope).
- Hoisting:** JS માં વેરિએબલ અને ફંક્શન ડિક્લેરેશનને કોડ રન થતાં પહેલાં તેમના સ્કોપમાં ટોચ પર (Top) લઈ જવામાં આવે છે. var હોઇસ્ટ થાય છે પણ undefined આપે છે, જ્યારે let અને const હોઇસ્ટ થાય છે પણ Temporal Dead Zone (TDZ) માં રહે છે.

૪. એરો ફંક્શન્સ (Arrow Functions)

ES6 માં આવેલી ફંક્શન લખવાની ટૂંકી અને આધુનિક રીત.

```
// નોર્મલ ફંક્શન
function add(a, b) { return a + b; }
```

```
// એરો ફંક્શન
const addArrow = (a, b) => a + b;
```

૫. DOM મેનિપ્યુલેશન (DOM Manipulation)

DOM (Document Object Model) દ્વારા JavaScript આખા HTML પેજને બદલી શકે છે.

- સિલેક્ટર:** document.getElementById(), document.querySelector()
- કન્ટેન્ટ બદલવું:** .innerHTML, .innerText
- સ્ટાઇલ બદલવી:** element.style.color = "red";

૬. ઇવેન્ટ્સ અને ઇવેન્ટ લિસનર્સ (Events & Event Listeners)

વેબ પેજ પર યુઝર દ્વારા થતી એક્ટિવિટી (જેમ કે ક્લિક, કી-પ્રેસ) ને ઇવેન્ટ કહેવાય છે.

```
let btn = document.querySelector("#myButton");
btn.addEventListener("click", function() {
  alert("બટન ક્લિક થયું!");
});
```

૭. સ્ટ્રિક્ટ મોડ (Strict Mode)

કોડની શરૂઆતમાં "use strict"; લખવાથી JS વધુ કડક બને છે. તે નાની ભૂલો પકડી પાડે છે અને કોડને વધુ સિક્યોર બનાવે છે (દા.ત. વેરિએબલ ડિક્લેરેશનમાં var નો વગર ડાયરેક્ટ વાપરી શકાતો નથી).

ભાગ ૩: એડવાન્સ લેવલ કોન્સેપ્ટ્સ (Advanced Level Concepts)

૧. એસિંક્રોનસ જાવાસ્ક્રિપ્ટ (Asynchronous JavaScript)

JS સિંગલ-થ્રેડેડ (એક સમયે એક જ કામ કરતી) લેંગ્વેજ છે, પણ તે લાંબો સમય લેતા કામો (જેમ કે API કોલ) એસિંક્રોનસ રીતે બેકગ્રાઉન્ડમાં કરી શકે છે.

- Callbacks:** એક ફંક્શન બીજા ફંક્શનની અંદર આર્ગ્યુમેન્ટ તરીકે પાસ થાય. (વધુ પડતા કોલબેકથી

Callback Hell બને છે).

- **Promises:** એસિંક્રોનસ ઓપરેશન સફળ (Resolve) થશે કે નિષ્ફળ (Reject) તે હેન્ડલ કરે છે. `.then()` અને `.catch()` નો ઉપયોગ થાય છે.
- **Async/Await:** પ્રોમિસ હેન્ડલ કરવાની સૌથી સરળ અને આધુનિક રીત છે. તે એસિંક્રોનસ કોડને સિંક્રોનસ જેવો લુક આપે છે.

```
// Async/Await ઉદાહરણ
async function fetchData() {
  try {
    let response = await fetch('https://api.example.com/data');
    let data = await response.json();
    console.log(data);
  } catch (error) {
    console.error("ભૂલ આવી: ", error);
  }
}
```

૨. ક્લોઝર્સ (Closures)

જ્યારે કોઈ ઇનર (અંદરનું) ફંક્શન તેના આઉટર (બહારના) ફંક્શનના વેરિએબલ્સને બહારનું ફંક્શન બંધ થઈ ગયા પછી પણ યાદ રાખે, ત્યારે તેને ક્લોઝર કહેવાય છે. આ ડેટા પ્રાઇવસી માટે ખૂબ ઉપયોગી છે.

૩. પ્રોટોટાઇપ અને ઇનહેરિટન્સ (Prototypes & Inheritance)

JS માં દરેક ઓબ્જેક્ટ પાસે એક ઇનબિલ્ટ પ્રોપર્ટી હોય છે જેને **Prototype** કહે છે. એક ઓબ્જેક્ટ બીજા ઓબ્જેક્ટના ફીચર્સ અને પ્રોપર્ટીઝ વારસામાં (Inherit) મેળવી શકે તે માટે પ્રોટોટાઇપલ ઇનહેરિટન્સ વપરાય છે.

૪. ક્લાસ અને OOPs (Classes & Object-Oriented Programming)

ES6 માં ક્લાસ કીવર્ડ આવ્યો જે ઓબ્જેક્ટ બનાવવા માટેની બ્લુપ્રિન્ટ પ્રોવાઇડ કરે છે. તેમાં `constructor`, `extends` (ઇનહેરિટન્સ માટે) અને `super()` વપરાય છે.

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `હેલો, હું ${this.name} છું.`;
  }
}
let user = new Person("દિવ્યેશ");
console.log(user.greet());
```

૫. એરર હેન્ડલિંગ (Error Handling - Try...Catch)

કોડ રન થતી વખતે કેશ ન થઈ જાય તે માટે ભૂલોને પકડવા માટે `try...catch...finally` બ્લોક વપરાય છે.

૬. મોડ્યુલ્સ (Modules - Export & Import)

મોટા પ્રોજેક્ટ્સમાં કોડને જુદી જુદી ફાઇલોમાં વહેંચવા માટે મોડ્યુલ્સ વપરાય છે. એક ફાઇલમાંથી `export` કરી બીજી ફાઇલમાં `import` કરાય છે.

૭. મેમરી મેનેજમેન્ટ અને ગાર્બેજ કલેક્શન (Memory Management)

JS ઓટોમેટિકલી મેમરી એલોકેટ કરે છે અને જ્યારે ઓબ્જેક્ટનો ઉપયોગ બંધ થઈ જાય, ત્યારે **Garbage Collector** (Mark-and-Sweep Algorithm દ્વારા) તે મેમરીને આપોઆપ ખાલી કરી દે છે.

ખૂબ ખૂબ અભિનંદન! તમે હવે JavaScript ની બેઝિક (Novel) થી લઈને પ્રો (Ninja) લેવલ સુધીના રોડમેપ